

Cahier des Charges – DPP Platform

Version : 1.0

Date : Janvier 2026

Vue d'ensemble du projet :

La **Digital Product Passport (DPP) Platform** est une solution en ligne permettant aux entreprises de créer, gérer et auditer les **Digital Product Passports (DPP)** en conformité avec les standards internationaux de durabilité. La plateforme exploite l'**AI for Good** et s'aligne avec la stratégie **SDG 2030** de l'**ITU** (International Telecommunication Union).

1. Objectifs

- **Frontend** : Fournir une plateforme conviviale et dynamique permettant aux utilisateurs de créer, gérer et auditer leurs **Digital Product Passports (DPP)**.
- **Backend** : Développer un système backend sécurisé et scalable qui gère l'authentification des utilisateurs, la création des DPP, l'audit des DPP, et prévoir un espace pour l'intégration future des **modèles d'IA** pour la vérification de la conformité et l'enrichissement des données.
- **Intégration IA** : Mettre en place des modèles d'IA pour l'évaluation de la conformité et l'enrichissement des DPP (intégration future).

2. Architecture Technique

2.1 Architecture du Frontend

Le frontend est développé avec **React** et **Vite** pour des performances élevées et des temps de build rapides.

Composants :

- **LandingPage** : Section marketing dynamique avec un accent sur les **SDG** et un chatbot interactif.
- **SignInForm** : Page de connexion, stockage du token **JWT** dans **localStorage** pour la persistance de la session.
- **SignUpForm** : Formulaire d'inscription pour les nouveaux utilisateurs, intégré avec les **API d'authentification backend**.
- **UserDashboard** : Affiche le nombre de DPP, les DPP récents et les résultats d'audit pour les utilisateurs authentifiés.
- **DppForm** : Formulaire de création de DPP, interactif avec le backend pour soumettre les données du produit.

- **AuditResults** : Formulaire de soumission de DPP pour audit, affichage des résultats générés par l'IA.
- **ProtectedRoute** : Composant pour protéger certaines routes nécessitant une authentification avant l'accès.

Pages :

- **/index** : Page d'accueil (Landing Page).
- **/signin** : Page de connexion.
- **/signup** : Page d'inscription.
- **/user-dashboard** : Affiche le tableau de bord personnalisé de l'utilisateur.
- **/dpp-create** : Page de création de DPP.
- **/audit** : Page des résultats d'audit.

Communication API :

- **api.js** pour les appels API centralisés (authentification, création de DPP, audit).
- **Auth** : Authentification des utilisateurs (connexion/inscription) gérée par les routes backend (**/auth/register** et **/auth/login**).
- **DPP Creation** : Création de DPP via la route **/dpp**.
- **Audit** : Téléchargement de fichiers et évaluation IA via la route **/audit**.

2.2 Architecture du Backend

Le backend est développé avec **FastAPI** (Python) et **SQLAlchemy** pour ORM et **JWT** pour l'authentification des utilisateurs.

Composants Clés :

- **main.py** : Point d'entrée, configure CORS, initialise l'application FastAPI et inclut les routeurs pour les différentes routes.
- **security.py** : Implémente le hachage des mots de passe, la génération de tokens et la validation de l'authentification des utilisateurs.
- **admin.py** : Protège les routes admin via un jeton **X-Admin-Token**.
- **session.py** : Gestion des sessions SQLAlchemy.
- **models.py** : Contient les modèles SQLAlchemy pour **User** et **DPP** avec un support **JSONB**.
- **schemas.py** : Schémas Pydantic pour la validation des données (par exemple, schéma DPP, schéma utilisateur).
- **auth.py** : Gère l'inscription et la connexion des utilisateurs.
- **dpp_creation.py** : Gère la création et la liste des DPP.
- **audit.py** : Gère l'upload de fichiers DPP pour audit, lié aux modèles IA pour l'évaluation de la conformité.

Intégration IA (Future) :

- **compliance_model.py** : Modèle IA de conformité (placeholders pour l'intégration future).
- **transformer_model.py** : Modèle IA d'enrichissement des données DPP.

Base de données :

- **PostgreSQL** : Utilisé pour stocker les données utilisateurs, DPP, et résultats d'audit.
- **SQLAlchemy** : ORM pour interagir avec la base de données.

Authentification :

- **JWT** : Utilisé pour authentifier les utilisateurs et sécuriser l'accès aux routes spécifiques.

2.3 Intégration IA

- **compliance_model.py** : Évaluation de la conformité des DPP (réponse simulée pour l'instant).
- **transformer_model.py** : Enrichissement des DPP via des modèles transformer (réponse simulée pour l'instant).

3. Fonctionnalités Clés

3.1 Gestion des Utilisateurs

- **Inscription et Connexion** :
 - Inscription d'un nouvel utilisateur avec un mot de passe haché et retour d'un token d'authentification.
 - Connexion de l'utilisateur avec validation du mot de passe et retour d'un token d'authentification.
 - Utilisation de **tokens JWT** pour sécuriser les routes.

3.2 Gestion des DPP

- **Création de DPP** :
 - L'utilisateur peut créer un **Digital Product Passport** en soumettant les détails du produit.
 - Les DPP sont stockés dans la base de données avec les métadonnées et le payload JSONB.
- **Liste des DPP** :
 - L'utilisateur peut lister tous les DPPs qu'il a créés ou auxquels il a accès.

3.3 Fonctionnalité d'Audit

- **Upload de DPP pour Audit :**
 - L'utilisateur peut uploader un fichier **DPP** (par exemple, JSON) pour vérification de la conformité via le système d'IA.
 - Les modèles IA génèrent une évaluation de conformité et un rapport d'audit.

3.4 Admin Panel

- Les administrateurs peuvent lister tous les utilisateurs inscrits et gérer les données de la plateforme. Les routes sont protégées et accessibles uniquement par des utilisateurs admins.

4. Sécurité et Conformité

4.1 Authentification:

- **Authentification JWT :**
 - **Inscription** : Crée un nouvel utilisateur avec un mot de passe haché.
 - **Connexion** : Authentifie l'utilisateur et retourne un token d'accès.
 - **Routes protégées** : Accès sécurisé aux routes avec un token JWT valide (tableau de bord de l'utilisateur, création de DPP, routes d'audit).

4.2 Gestion des Rôles:

- **Rôles Admin :**

L'administrateur peut accéder et gérer toutes les données des utilisateurs et des DPP.

4.3 Protection des Données:

- **Conformité GDPR :**

La plateforme garantit que les données utilisateurs sont traitées conformément aux réglementations de protection des données (notamment le **GDPR**).

5. Tests et Assurance Qualité

5.1 Tests Unitaires:

- Test de base inclus dans **test_dpp.py** pour vérifier le point de terminaison de la vérification de l'état de santé.

5.2 Tests Futurs:

- **Test Authentification** : Vérifier le flux d'authentification JWT.

- **Test Création et Liste des DPP** : Assurer que les DPP peuvent être créés et listés correctement.
- **Test de l'Audit** : Vérifier que les DPP uploadés sont correctement évalués par l'IA.

6. Configuration du Projet et Exécution

6.1 Configuration du Frontend:

Naviguer dans le répertoire **frontend** :

```
cd frontend
npm install
npm run dev
```

-
- Le frontend sera accessible à **http://localhost:3000**.

6.2 Configuration du Backend:

Naviguer dans le répertoire **backend** :

```
cd backend
.venv\Scripts\activate # Activer l'environnement virtuel (Windows)
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

-
- Le backend sera accessible à **http://127.0.0.1:8000**.

7. Développement Futur

- **Modèles d'IA** : Implémentation des véritables modèles de conformité et d'enrichissement des DPP pour remplacer les modèles simulés actuels.
- **Fonctionnalités Admin** : Amélioration du panel d'administration pour un contrôle plus fin des utilisateurs et des données.
- **Scalabilité** : Optimisation du frontend et du backend pour gérer de plus grands ensembles de données et plus d'utilisateurs.

Conclusion:

Ce **Cahier des Charges** fournit un aperçu complet de la **DPP Platform**, détaillant l'architecture du frontend et du backend, les fonctionnalités clés et les exigences de sécurité. Cette base

permettra au projet de progresser avec une solide structure et un objectif clair, en ligne avec les **SDG** et la stratégie **AI for Good**.